

API REST de Moodle

Referencia Completa + Guía Práctica

Postman · NestJS · Más de 200 funciones

```
POST https://tu-moodle.com/webservice/rest/server.php?wstoken=TOKEN&wsfunction;=FUNCION&moodlewsrestformat=json
```

Lectura

Solo consulta datos

Escritura

Crea o modifica datos

Índice de Contenidos

PARTE A — REFERENCIA DE FUNCIONES

- [1. Autenticación y Sitio](#)
- [2. Usuarios](#)
- [3. Cursos y Categorías](#)
- [4. Matrículas y Roles](#)
- [5. Grupos y Agrupaciones](#)
- [6. Calificaciones](#)
- [7. Archivos](#)
- [8. Mensajes y Notificaciones](#)
- [9. Calendario y Eventos](#)
- [10. Tareas \(mod_assign\)](#)
- [11. Cuestionarios \(mod_quiz\)](#)
- [12. Foros \(mod_forum\)](#)
- [13. SCORM](#)
- [14. Completación, Comentarios y Notas](#)
- [15. Recursos, Wiki, Glosario y Otras Actividades](#)

PARTE B — GUÍA POSTMAN

- [B.1 Configurar Variables de Entorno](#)
- [B.2 Autenticación — Obtener Token](#)
- [B.3 GET: Info del Sitio](#)
- [B.4 POST: Crear Usuario](#)
- [B.5 POST: Matricular Usuario en Curso](#)
- [B.6 GET: Listar Usuarios Matriculados](#)
- [B.7 POST: Crear Evento de Calendario](#)
- [B.8 GET: Calificaciones de Usuario](#)
- [B.9 Pre-request Script Dinámico](#)
- [B.10 Tests Automáticos en Postman](#)
- [B.11 Manejo de Errores](#)

PARTE C — GUÍA NESTJS

- [C.1 Setup e Instalación](#)
- [C.2 MoodleService — Servicio Central](#)
- [C.3 Módulo de Usuarios](#)
- [C.4 Módulo de Cursos](#)
- [C.5 Módulo de Matrículas](#)
- [C.6 Módulo de Calificaciones](#)
- [C.7 Módulo de Mensajería](#)
- [C.8 Manejo de Errores con Filtros](#)
- [C.9 Interceptor de Logs](#)
- [C.10 Ejemplo Completo: Controlador REST](#)

PARTE A — Referencia de Funciones

1. Autenticación y Sitio

Función REST	Tipo	Descripción
core_webservice_get_site_info	Lectura	Obtiene información general del sitio Moodle (nombre, versión, idioma, capacidades del usuario). Útil como primer llamado para validar token.
core_auth_request_password_reset	Escritura	Solicita restablecimiento de contraseña por correo para un usuario dado su email o nombre de usuario.
core_auth_confirm_age_digital_consent	Escritura	Confirma el consentimiento de edad digital para un usuario menor.
core_auth_is_age_digital_consent_verification_enabled	Lectura	Verifica si la comprobación de consentimiento de edad digital está habilitada en el sitio.
core_auth_resend_confirmation_email	Escritura	Reenvía el correo de confirmación de cuenta a un usuario.

2. Usuarios

Función REST	Tipo	Descripción
core_user_create_users	Escritura	Crea uno o múltiples usuarios en Moodle. Requiere nombre, apellido, email, nombre de usuario y contraseña.
core_user_update_users	Escritura	Actualiza los datos de uno o varios usuarios existentes (nombre, email, descripción, ciudad, país, etc.).
core_user_delete_users	Escritura	Elimina (suspende) usuarios del sistema por su ID.
core_user_get_users	Lectura	Busca usuarios aplicando criterios flexibles como email, nombre de usuario, nombre, apellido, etc.
core_user_get_users_by_field	Lectura	Obtiene usuarios filtrando por un campo específico (id, idnumber, username, email).
core_user_get_course_user_profiles	Lectura	Retorna el perfil completo de usuarios dentro de un curso específico.
core_user_get_user_preferences	Lectura	Obtiene las preferencias de configuración de un usuario (notificaciones, idioma, etc.).
core_user_set_user_preferences	Escritura	Establece o actualiza preferencias para un usuario.
core_user_update_picture	Escritura	Actualiza la imagen de perfil de un usuario usando un archivo ya subido.
core_user_remove_user_device	Escritura	Elimina un dispositivo móvil registrado para notificaciones push de un usuario.
core_user_add_user_device	Escritura	Registra un nuevo dispositivo móvil (token push) para notificaciones de un usuario.

Función REST	Tipo	Descripción
core_user_add_user_private_files	Escritura	Agrega archivos a la zona de archivos privados del usuario.
core_user_get_private_files_info	Lectura	Retorna información sobre el espacio de archivos privados del usuario (usado, disponible).
core_user_agree_site_policy	Escritura	Registra que el usuario aceptó la política del sitio.
core_user_view_user_list	Escritura	Registra evento de visualización de lista de usuarios en un curso.
core_user_view_user_profile	Escritura	Registra evento de visualización del perfil de un usuario.
core_user_search_identity	Lectura	Busca usuarios por su identidad (nombre, email) para autocompletado.

3. Cursos y Categorías

Función REST	Tipo	Descripción
core_course_create_courses	Escritura	Crea uno o múltiples cursos con todos sus parámetros: nombre, shortname, categoría, formato, fechas, visibilidad, etc.
core_course_update_courses	Escritura	Actualiza parámetros de cursos existentes.
core_course_delete_courses	Escritura	Elimina permanentemente uno o varios cursos por ID.
core_course_get_courses	Lectura	Obtiene una lista de cursos (todos o filtrados por IDs).
core_course_get_courses_by_field	Lectura	Obtiene cursos filtrando por campo específico: id, ids, shortname, idnumber, category.
core_course_search_courses	Lectura	Busca cursos por texto libre con soporte de paginación.
core_course_get_contents	Lectura	Obtiene el contenido completo de un curso: secciones, módulos y recursos.
core_course_get_course_module	Lectura	Obtiene información de un módulo de actividad específico por su ID.
core_course_get_course_module_by_instance	Lectura	Obtiene el módulo de curso dado el nombre del módulo y su ID de instancia.
core_course_duplicate_course	Escritura	Duplica un curso existente con su contenido.
core_course_import_course	Escritura	Importa contenido de un curso a otro.
core_course_view_course	Escritura	Registra el evento de visualización de un curso (usado para completion y logs).
core_course_get_recent_courses	Lectura	Obtiene los cursos visitados recientemente por el usuario.
core_course_get_enrolled_courses_by_timeline_classification	Lectura	Obtiene los cursos del usuario clasificados por línea de tiempo: inprogress, future, past, starred, hidden.
core_course_set_favourite_courses	Escritura	Marca o desmarca cursos como favoritos para el usuario.

Función REST	Tipo	Descripción
core_course_get_categories	Lectura	Obtiene categorías de cursos con filtros opcionales.
core_course_create_categories	Escritura	Crea nuevas categorías de cursos.
core_course_update_categories	Escritura	Actualiza categorías de cursos existentes.
core_course_delete_categories	Escritura	Elimina categorías de cursos.

4. Matrículas y Roles

Función REST	Tipo	Descripción
enrol_manual_enrol_users	Escritura	Matricula manualmente usuarios en cursos asignando un rol específico.
enrol_manual_unenrol_users	Escritura	Desmatricula usuarios de cursos (matrícula manual).
enrol_self_enrol_user	Escritura	Matricula al usuario autenticado en un curso con auto-matrícula.
enrol_self_get_instance_info	Lectura	Obtiene información de una instancia de auto-matrícula en un curso.
enrol_guest_get_instance_info	Lectura	Obtiene información de la instancia de matrícula de invitado de un curso.
core_enrol_get_enrolled_users	Lectura	Obtiene la lista de usuarios matriculados en un curso con sus datos de perfil.
core_enrol_get_users_courses	Lectura	Obtiene todos los cursos en los que está matriculado un usuario.
core_enrol_get_course_enrolment_methods	Lectura	Retorna los métodos de matrícula habilitados en un curso.
core_enrol_search_users	Lectura	Busca usuarios matriculados en un curso con filtros.
core_role_assign_roles	Escritura	Asigna uno o más roles a usuarios en un contexto específico.
core_role_unassign_roles	Escritura	Quita la asignación de roles a usuarios en un contexto.

5. Grupos y Agrupaciones

Función REST	Tipo	Descripción
core_group_create_groups	Escritura	Crea uno o varios grupos dentro de un curso.
core_group_update_groups	Escritura	Actualiza los datos de grupos existentes.
core_group_delete_groups	Escritura	Elimina grupos por ID.
core_group_get_groups	Lectura	Obtiene grupos de un curso por sus IDs.
core_group_get_course_groups	Lectura	Obtiene todos los grupos de un curso.
core_group_get_course_user_groups	Lectura	Obtiene grupos de un curso a los que pertenece un usuario.
core_group_add_group_members	Escritura	Agrega miembros a grupos.
core_group_delete_group_members	Escritura	Elimina miembros de grupos.
core_group_get_group_members	Lectura	Obtiene los miembros de grupos específicos.
core_group_create_groupings	Escritura	Crea agrupaciones en un curso.
core_group_update_groupings	Escritura	Actualiza agrupaciones existentes.

Función REST	Tipo	Descripción
core_group_delete_groupings	Escritura	Elimina agrupaciones.
core_group_get_groupings	Lectura	Obtiene agrupaciones por ID.
core_group_get_course_groupings	Lectura	Obtiene todas las agrupaciones de un curso.
core_group_assign_grouping	Escritura	Asigna grupos a agrupaciones.
core_group_unassign_grouping	Escritura	Quita grupos de agrupaciones.

6. Calificaciones

Función REST	Tipo	Descripción
core_grades_update_grades	Escritura	Actualiza calificaciones de usuarios en un módulo de actividad específico.
core_grades_get_grades	Lectura	Obtiene calificaciones de usuarios en actividades de un curso.
gradereport_user_get_grade_items	Lectura	Obtiene ítems de calificaciones de un usuario en un curso.
gradereport_user_get_grades_table	Lectura	Retorna la tabla de calificaciones completa para un usuario en un curso.
gradereport_overview_get_course_grades	Lectura	Obtiene las calificaciones generales de cursos para un usuario.
core_grading_get_definitions	Lectura	Obtiene las definiciones de calificación (rúbricas, guías de calificación) de actividades.
core_grading_save_definitions	Escritura	Guarda definiciones de calificación avanzada.

7. Archivos

Función REST	Tipo	Descripción
core_files_get_files	Lectura	Lista archivos de un área específica de almacenamiento de Moodle (contextid, component, filearea, itemid).
core_files_upload	Escritura	Sube un archivo a un área de borrador del repositorio de archivos.
core_files_delete_draft_files	Escritura	Elimina archivos del área de borrador.
core_files_get_unused_draft_itemid	Lectura	Obtiene un itemid de área de borrador sin usar para subir archivos nuevos.

8. Mensajes y Notificaciones

Función REST	Tipo	Descripción
core_message_send_messages	Escritura	Envía mensajes privados entre usuarios.
core_message_send_messages_to_conversation	Escritura	Envía un mensaje a una conversación existente.
core_message_get_messages	Lectura	Obtiene mensajes enviados o recibidos por un usuario con filtros.
core_message_get_conversations	Lectura	Obtiene todas las conversaciones de un usuario.
core_message_get_conversation	Lectura	Obtiene una conversación específica por ID.
core_message_get_conversation_members	Lectura	Obtiene miembros de una conversación.
core_message_get_conversation_messages	Lectura	Obtiene mensajes de una conversación específica.
core_message_create_contact_request	Escritura	Crea una solicitud de contacto entre usuarios.
core_message_confirm_contact_request	Escritura	Acepta una solicitud de contacto.
core_message_decline_contact_request	Escritura	Rechaza una solicitud de contacto.
core_message_get_contact_requests	Lectura	Obtiene las solicitudes de contacto de un usuario.
core_message_block_user	Escritura	Bloquea a un usuario para que no pueda enviar mensajes.
core_message_unblock_user	Escritura	Desbloquea a un usuario bloqueado.
core_message_get_blocked_users	Lectura	Obtiene la lista de usuarios bloqueados.
core_message_delete_message	Escritura	Elimina un mensaje del historial del usuario.
core_message_mark_message_read	Escritura	Marca un mensaje como leído.

Función REST	Tipo	Descripción
core_message_mark_all_messages_as_read	Escritura	Marca todos los mensajes de una conversación como leídos.
core_message_get_unread_conversations_count	Lectura	Obtiene el número de conversaciones con mensajes no leídos.
core_message_search_contacts	Lectura	Busca contactos disponibles para mensajería.
message_popup_get_popup_notifications	Lectura	Obtiene notificaciones emergentes (popup) de un usuario.
message_popup_get_unread_popup_notification_count	Lectura	Obtiene el conteo de notificaciones popup no leídas.

9. Calendario y Eventos

Función REST	Tipo	Descripción
core_calendar_create_calendar_events	Escritura	Crea eventos en el calendario (personales, de curso, de grupo o del sitio).
core_calendar_update_event_start_day	Escritura	Actualiza la fecha de inicio de un evento del calendario.
core_calendar_delete_calendar_events	Escritura	Elimina eventos del calendario por ID.
core_calendar_get_calendar_events	Lectura	Obtiene eventos del calendario con filtros por tipo, usuario o curso.
core_calendar_get_action_events_by_course	Lectura	Obtiene eventos de acción pendientes en un curso para el usuario.
core_calendar_get_action_events_by_courses	Lectura	Obtiene eventos de acción pendientes en múltiples cursos.
core_calendar_get_action_events_by_timesort	Lectura	Obtiene eventos de acción ordenados por fecha con paginación.
core_calendar_get_calendar_month_view	Lectura	Obtiene la vista mensual del calendario.
core_calendar_get_calendar_day_view	Lectura	Obtiene la vista diaria del calendario.
core_calendar_get_calendar_upcoming_view	Lectura	Obtiene la vista de próximos eventos del calendario.

10. Tareas (mod_assign)

Función REST	Tipo	Descripción
mod_assign_get_assignments	Lectura	Obtiene todas las tareas de cursos especificados con sus configuraciones.
mod_assign_get_grades	Lectura	Obtiene las calificaciones de participantes en una tarea.
mod_assign_save_grade	Escritura	Guarda la calificación y el feedback de un usuario en una tarea.
mod_assign_save_grades	Escritura	Guarda calificaciones de múltiples usuarios en una tarea a la vez.
mod_assign_get_submissions	Lectura	Obtiene las entregas de participantes en una o varias tareas.
mod_assign_submit_for_grading	Escritura	Envía una tarea para calificación (acción del estudiante).
mod_assign_save_submission	Escritura	Guarda la entrega del estudiante (texto en línea o archivos).
mod_assign_revert_submissions_to_draft	Escritura	Revierte entregas al estado de borrador.
mod_assign_lock_submissions	Escritura	Bloquea las entregas de participantes.
mod_assign_unlock_submissions	Escritura	Desbloquea entregas de participantes.
mod_assign_get_submission_statuses	Lectura	Obtiene el estado de entrega de un usuario en una tarea.
mod_assign_get_participant	Lectura	Obtiene la información de un participante en una tarea.
mod_assign_get_user_flags	Lectura	Obtiene los flags de usuario en una tarea (extensiones, bloqueado, etc.).
mod_assign_set_user_flags	Escritura	Establece flags de usuario en una tarea.
mod_assign_start_submission	Escritura	Inicia una nueva entrega para el usuario.
mod_assign_view_assign	Escritura	Registra el evento de visualización de una tarea.

11. Cuestionarios (mod_quiz)

Función REST	Tipo	Descripción
mod_quiz_get_quizzes_by_courses	Lectura	Obtiene todos los cuestionarios de cursos específicos.
mod_quiz_get_user_attempts	Lectura	Obtiene los intentos de un usuario en un cuestionario.
mod_quiz_get_attempt_data	Lectura	Obtiene los datos de un intento incluyendo preguntas.
mod_quiz_get_attempt_summary	Lectura	Obtiene el resumen de un intento de cuestionario.
mod_quiz_get_attempt_review	Lectura	Obtiene la revisión completa de un intento finalizado.
mod_quiz_start_attempt	Escritura	Inicia un nuevo intento en un cuestionario.
mod_quiz_process_attempt	Escritura	Procesa las respuestas de una página de un intento y navega entre páginas.

Función REST	Tipo	Descripción
mod_quiz_finish_attempt	Escritura	Finaliza y envía un intento de cuestionario.
mod_quiz_get_quiz_access_information	Lectura	Obtiene información de acceso al cuestionario: intentos restantes, reglas de acceso.
mod_quiz_get_attempt_access_information	Lectura	Obtiene información de acceso a un intento específico.
mod_quiz_view_quiz	Escritura	Registra el evento de visualización de un cuestionario.
mod_quiz_view_attempt	Escritura	Registra el evento de visualización de un intento.
mod_quiz_view_attempt_review	Escritura	Registra el evento de visualización de la revisión de un intento.

12. Foros (mod_forum)

Función REST	Tipo	Descripción
mod_forum_get_forums_by_courses	Lectura	Obtiene todos los foros de cursos específicos con su configuración.
mod_forum_get_forum_discussions	Lectura	Obtiene las discusiones de un foro con paginación.
mod_forum_get_forum_discussion_posts	Lectura	Obtiene los posts/respuestas de una discusión.
mod_forum_get_discussion_post	Lectura	Obtiene un post específico de una discusión.
mod_forum_add_discussion	Escritura	Crea una nueva discusión en un foro.
mod_forum_add_discussion_post	Escritura	Agrega una respuesta a una discusión existente.
mod_forum_update_discussion_post	Escritura	Actualiza el contenido de un post existente.
mod_forum_delete_post	Escritura	Elimina un post de una discusión.
mod_forum_can_add_discussion	Lectura	Verifica si el usuario puede agregar una nueva discusión en un foro.
mod_forum_set_subscription_state	Escritura	Suscribe o desuscribe al usuario de un foro.
mod_forum_toggle_favourite_state	Escritura	Marca o desmarca una discusión como favorita.
mod_forum_set_lock_state	Escritura	Bloquea o desbloquea una discusión.
mod_forum_set_pin_state	Escritura	Fija o desfija una discusión en el foro.
mod_forum_view_forum	Escritura	Registra el evento de visualización de un foro.

13. SCORM

Función REST	Tipo	Descripción
mod_scorm_get_scorms_by_courses	Lectura	Obtiene todos los objetos SCORM de cursos específicos.
mod_scorm_get_scorm_access_information	Lectura	Obtiene información de acceso a un SCORM (intentos restantes, reglas).
mod_scorm_get_scorm_attempt_count	Lectura	Obtiene el número de intentos de un usuario en un SCORM.
mod_scorm_get_scorm_sco_tracks	Lectura	Obtiene los datos de seguimiento (tracks) de un SCO en un intento.
mod_scorm_get_scorm_scoes	Lectura	Obtiene los SCOs (Sharable Content Objects) de un SCORM.
mod_scorm_get_scorm_user_data	Lectura	Obtiene los datos de usuario para todos los SCOs de un intento.
mod_scorm_insert_scorm_tracks	Escritura	Inserta/actualiza datos de seguimiento cmi.* en un SCO.
mod_scorm_launch_sco	Escritura	Registra el evento de lanzamiento de un SCO.
mod_scorm_view_scorm	Escritura	Registra el evento de visualización de un SCORM.

14. Completación, Comentarios y Notas

Función REST	Tipo	Descripción
core_completion_get_activities_completion_status	Lectura	Obtiene el estado de completación de todas las actividades de un curso para un usuario.
core_completion_get_course_completion_status	Lectura	Obtiene el estado de completación del curso completo para un usuario.
core_completion_update_activity_completion_status_manually	Escritura	Actualiza manualmente el estado de completación de una actividad para un usuario.
core_completion_mark_course_self_completed	Escritura	Marca el curso como completado manualmente por el propio usuario.
core_comment_add_comments	Escritura	Agrega comentarios a un área de comentarios (actividad, usuario, etc.).
core_comment_delete_comments	Escritura	Elimina comentarios por ID.
core_comment_get_comments	Lectura	Obtiene comentarios de un área específica.
core_notes_create_notes	Escritura	Crea notas (personales, de curso o del sitio) para usuarios.
core_notes_update_notes	Escritura	Actualiza notas existentes.
core_notes_delete_notes	Escritura	Elimina notas por ID.
core_notes_get_notes	Lectura	Obtiene notas por sus IDs.
core_notes_get_course_notes	Lectura	Obtiene las notas de un curso.

Función REST	Tipo	Descripción
core_rating_get_item_ratings	Lectura	Obtiene las valoraciones (ratings) de un elemento de actividad.
core_rating_add_rating	Escritura	Agrega o actualiza una valoración de un elemento.
core_badges_get_user_badges	Lectura	Obtiene las insignias (badges) obtenidas por un usuario.

15. Recursos, Wiki, Glosario y Otras Actividades

Función REST	Tipo	Descripción
mod_resource_get_resources_by_courses	Lectura	Obtiene archivos/recursos (mod_resource) de cursos.
mod_page_get_pages_by_courses	Lectura	Obtiene páginas (mod_page) de cursos.
mod_url_get_urls_by_courses	Lectura	Obtiene URLs (mod_url) de cursos.
mod_folder_get_folders_by_courses	Lectura	Obtiene carpetas (mod_folder) de cursos.
mod_book_get_books_by_courses	Lectura	Obtiene libros (mod_book) de cursos.
mod_wiki_get_wikis_by_courses	Lectura	Obtiene wikis de cursos.
mod_wiki_get_pages_list	Lectura	Obtiene la lista de páginas de una wiki.
mod_wiki_get_page_contents	Lectura	Obtiene el contenido de una página wiki.
mod_wiki_new_page	Escritura	Crea una nueva página en una wiki.
mod_wiki_edit_page	Escritura	Edita una página wiki existente.
mod_glossary_get_glossaries_by_courses	Lectura	Obtiene glosarios de cursos.
mod_glossary_get_entries_by_letter	Lectura	Obtiene entradas de glosario filtradas por letra.
mod_glossary_add_entry	Escritura	Agrega una nueva entrada al glosario.
mod_glossary_delete_entry	Escritura	Elimina una entrada del glosario.
mod_choice_get_choices_by_courses	Lectura	Obtiene consultas (choices) de cursos.
mod_choice_submit_choice_response	Escritura	Envía una respuesta en una consulta.
mod_choice_get_choice_results	Lectura	Obtiene los resultados de una consulta.
mod_feedback_get_feedbacks_by_courses	Lectura	Obtiene actividades de feedback de cursos.
mod_feedback_get_items	Lectura	Obtiene los ítems/preguntas de una actividad feedback.
mod_feedback_process_page	Escritura	Procesa una página de respuestas en un feedback.

Función REST	Tipo	Descripción
mod_survey_get_surveys_by_courses	Lectura	Obtiene encuestas de cursos.
mod_survey_submit_answers	Escritura	Envía respuestas a una encuesta.
mod_chat_get_chats_by_courses	Lectura	Obtiene chats de cursos.
mod_chat_send_chat_message	Escritura	Envía un mensaje a una sala de chat.
mod_workshop_get_workshops_by_courses	Lectura	Obtiene talleres (workshops) de cursos.
core_search_get_results	Lectura	Realiza búsqueda global en el sitio Moodle.

PARTE B — Guía Práctica: Postman

Postman permite probar todos los endpoints de Moodle de forma visual. Recomendamos crear una Collection llamada 'Moodle API' con una Environment que almacene el token y la URL base. Todos los llamados usan Content-Type: application/x-www-form-urlencoded.

B.1 — Configurar Variables de Entorno

En Postman ve a Environments > New, y crea las siguientes variables:

Environment Variables

Variable	Initial Value	Descripción
MOODLE_URL	https://tu-moodle.com	URL base sin barra final
MOODLE_TOKEN	abc123def456...	Token del servicio web
MOODLE_USER_ID	2	ID del usuario admin
MOODLE_COURSE_ID	10	ID de curso de prueba

B.2 — Autenticación: Obtener Token

Antes de usar la API debes obtener un token desde el endpoint de login:

POST — Obtener Token de Autenticación

URL: {{MOODLE_URL}}/login/token.php Método: POST Body (x-www-form-urlencoded): username = admin password = tuPasswordSegura service = moodle_mobile_app Respuesta exitosa: { "token": "abc123def456...", "privatetoken": null } Respuesta de error: { "error": "invalidlogin", "stacktrace": null }

B.3 — GET: Información del Sitio

GET — core_webservice_get_site_info

URL: {{MOODLE_URL}}/webservice/rest/server.php Método: GET Query Params: wstoken = {{MOODLE_TOKEN}} wsfunction = core_webservice_get_site_info moodlewsrestformat = json Respuesta: { "sitename": "Mi Plataforma", "username": "admin", "userid": 2, "release": "4.3.1+", "lang": "es", "functions": [...] }

B.4 — POST: Crear Usuario

POST — core_user_create_users

```
URL: {{MOODLE_URL}}/webservice/rest/server.php Método: POST Body (x-www-form-urlencoded):
wstoken = {{MOODLE_TOKEN}} wsfunction = core_user_create_users moodlewsrestformat = json
users[0][username] = juan.perez users[0][password] = Pass1234! users[0][firstname] = Juan
users[0][lastname] = Perez users[0][email] = juan@empresa.com users[0][auth] = manual
users[0][idnumber] = EMP001 users[0][lang] = es users[0][timezone] = America/La_Paz
users[0][mailformat] = 1 users[0][description] = Empleado de prueba users[0][city] = La Paz
users[0][country] = BO Respuesta: [{ "id": 42, "username": "juan.perez" }]
```

B.5 — POST: Matricular Usuario en Curso

POST — enrol_manual_enrol_users

```
URL: {{MOODLE_URL}}/webservice/rest/server.php Método: POST Body (x-www-form-urlencoded):
wstoken = {{MOODLE_TOKEN}} wsfunction = enrol_manual_enrol_users moodlewsrestformat = json
enrolments[0][roleid] = 5 // 5=estudiante, 3=editor, 4=profesor enrolments[0][userid] = 42
enrolments[0][courseid] = 10 enrolments[0][timestart] = 0 // 0 = sin restricción de inicio
enrolments[0][timeend] = 0 // 0 = sin fecha de fin enrolments[0][suspend] = 0 // 0 = activo, 1
= suspendido Respuesta exitosa: null (null = éxito, sin error = operación completada)
```

B.6 — GET: Usuarios Matriculados en un Curso

GET — core_enrol_get_enrolled_users

```
Query Params: wstoken = {{MOODLE_TOKEN}} wsfunction = core_enrol_get_enrolled_users
moodlewsrestformat = json courseid = {{MOODLE_COURSE_ID}} options[0][name] = withcapability
options[0][value] = mod/assign:submit options[1][name] = userfields options[1][value] =
id,fullname,email,roles options[2][name] = limitfrom options[2][value] = 0 options[3][name] =
limitnumber options[3][value] = 50 Respuesta: [ { "id": 42, "fullname": "Juan Perez", "email":
"juan@empresa.com", "roles": [{ "roleid": 5, "name": "Student" } ] } ]
```

B.7 — POST: Crear Evento de Calendario

POST — core_calendar_create_calendar_events

```
Body (x-www-form-urlencoded): wstoken = {{MOODLE_TOKEN}} wsfunction =
core_calendar_create_calendar_events moodlewsrestformat = json events[0][name] = Examen Final
events[0][description] = Evaluacion del modulo 3 events[0][format] = 1 events[0][courseid] =
10 events[0][groupid] = 0 events[0][userid] = 0 events[0][repeatid] = 0 events[0][eventtype] =
course // user|course|group|site events[0][timestart] = 1735689600 // Unix timestamp
events[0][timeduration] = 7200 // duración en segundos events[0][visible] = 1
events[0][sequence] = 1
```

B.8 — GET: Calificaciones de un Usuario

GET — gradereport_user_get_grade_items


```
Query Params: wstoken = {{MOODLE_TOKEN}} wsfunction = gradereport_user_get_grade_items
moodlewsrestformat = json courseid = 10 userid = 42 groupid = 0 Respuesta: { "usergrades": [{
"courseid": 10, "userid": 42, "gradeitems": [ { "id": 5, "itemname": "Tarea 1", "graderaw":
85.0, "grademax": 100.0, "percentageformatted": "85.00 %" } ] ] } }
```

B.9 — Pre-request Script: Token Dinámico

Agrega este script en la pestaña 'Pre-request Script' de tu Collection para renovar el token automáticamente en cada petición:

Pre-request Script (JavaScript)

```
// Obtiene token fresco antes de cada request const moodleUrl =
pm.environment.get('MOODLE_URL'); const username = pm.environment.get('MOODLE_ADMIN_USER');
const password = pm.environment.get('MOODLE_ADMIN_PASS'); pm.sendRequest({ url:
`${moodleUrl}/login/token.php`, method: 'POST', header: { 'Content-Type':
'application/x-www-form-urlencoded' }, body: { mode: 'urlencoded', urlencoded: [ { key:
'username', value: username }, { key: 'password', value: password }, { key: 'service', value:
'moodle_mobile_app' } ] } }, function(err, response) { if (err) { console.error('Error al
obtener token:', err); return; } const data = response.json(); if (data.token) {
pm.environment.set('MOODLE_TOKEN', data.token); console.log('Token renovado correctamente'); }
else { console.error('Error de autenticacion:', data.error); } });
```

B.10 — Tests Automáticos en Postman

Agrega estos tests en la pestaña 'Tests' para validar respuestas automáticamente:

Tests Script (JavaScript)

```
// 1. Verificar status 200 pm.test('Status 200', () => pm.response.to.have.status(200)); // 2.
Verificar que la respuesta es JSON válido pm.test('Es JSON', () => pm.response.to.be.json); //
3. Verificar que no hay error de Moodle pm.test('Sin error Moodle', () => { const body =
pm.response.json(); pm.expect(body).to.not.have.property('exception');
pm.expect(body).to.not.have.property('errorcode'); }); // 4. Guardar ID del recurso creado
(ej. usuario) const data = pm.response.json(); if (Array.isArray(data) && data[0] &&
data[0].id) { pm.environment.set('LAST_CREATED_ID', data[0].id); console.log('ID guardado:',
data[0].id); } // 5. Medir tiempo de respuesta pm.test('Responde en menos de 3s', () => {
pm.expect(pm.response.responseTime).to.be.below(3000); });
```

B.11 — Manejo de Errores de Moodle

Moodle siempre responde con HTTP 200 pero incluye campos de error en el JSON. Es fundamental verificar estos campos en cada respuesta:

Estructura de Error de Moodle (HTTP 200 con error)

```
// Moodle responde HTTP 200 incluso cuando hay un error. // Debes verificar el campo
'exception' o 'errorcode' en el body: { "exception": "moodle_exception", "errorcode":
"invalidtoken", "message": "Invalid token - token not found", "debuginfo": "Token abc123 not
found in DB" } // Códigos de error comunes: // invalidtoken – Token inválido o expirado //
accessdenied – Sin permisos para ejecutar la función // invalidparameter – Parámetro faltante
o con formato incorrecto // courseidnotfound – El curso no existe // useridnotfound – El
usuario no existe // nopermissions – Usuario sin capacidades requeridas
```

PARTE C — Guía Práctica: NestJS

Se construye un módulo MoodleModule reutilizable con un MoodleService central que maneja autenticación, formato de parámetros y errores. Cada dominio (usuarios, cursos, matrículas) tiene su propio servicio especializado.

C.1 — Setup e Instalación

Terminal — Instalar Dependencias

```
npm install @nestjs/axios axios @nestjs/config npm install --save-dev @types/node
```

.env

```
MOODLE_URL=https://tu-moodle.com MOODLE_TOKEN=abc123def456... MOODLE_ADMIN_USER=admin  
MOODLE_ADMIN_PASS=tuPassword
```

app.module.ts

```
import { Module } from '@nestjs/common'; import { HttpModule } from '@nestjs/axios'; import {  
  ConfigModule } from '@nestjs/config'; import { MoodleModule } from './moodle/moodle.module';  
@Module({ imports: [ ConfigModule.forRoot({ isGlobal: true }), HttpModule.register({ timeout:  
  10000, maxRedirects: 5 }), MoodleModule, ], }) export class AppModule {}
```

C.2 — MoodleService: Servicio Central

Este servicio centraliza todos los llamados a la API, maneja la serialización de parámetros anidados (arrays con índices) y parsea errores de Moodle:

moodle/moodle.service.ts

```
import { Injectable, HttpException, HttpStatus } from '@nestjs/common'; import { HttpService }
from '@nestjs/axios'; import { ConfigService } from '@nestjs/config'; import { firstValueFrom
} from 'rxjs'; @Injectable() export class MoodleService { private readonly url: string;
private readonly token: string; constructor( private http: HttpService, private config:
ConfigService, ) { this.url = config.get('MOODLE_URL'); this.token =
config.get('MOODLE_TOKEN'); } /** Llamado central a cualquier función de la API REST */ async
call(wsfunction: string, params: Record = {}): Promise { const body = new URLSearchParams({
wstoken: this.token, wsfunction, moodlewsrestformat: 'json', ...this.flatten(params), });
const { data } = await firstValueFrom( this.http.post(
`${this.url}/webservice/rest/server.php`, body.toString(), { headers: { 'Content-Type':
'application/x-www-form-urlencoded' } }, ) ); // Moodle responde HTTP 200 aunque haya error –
verificar campo exception if (data?.exception || data?.errorcode) { throw new HttpException( {
moodleError: data.errorcode, message: data.message }, HttpStatus.BAD_REQUEST, ); } return data
as T; } /** Aplana objetos anidados a formato Moodle: users[0][name]=Juan */ private
flatten(obj: any, prefix = ''): Record { return Object.entries(obj).reduce((acc, [key, val])
=> { const fullKey = prefix ? `${prefix}[${key}]` : key; if (Array.isArray(val)) {
val.forEach((item, i) => { if (typeof item === 'object' && item !== null) { Object.assign(acc,
this.flatten(item, `${fullKey}[${i}]`)); } else { acc[`${fullKey}[${i}]`] = String(item); }
}); } else if (typeof val === 'object' && val !== null) { Object.assign(acc, this.flatten(val,
fullKey)); } else { acc[fullKey] = String(val ?? ''); } return acc; }, {} as Record); } }
```

C.3 — Módulo de Usuarios

moodle/users.service.ts

```
import { Injectable } from '@nestjs/common'; import { MoodleService } from '../moodle.service';
export interface MoodleUser { id: number; username: string; firstname: string; lastname:
string; email: string; suspended: number; } export interface CreateUserDto { username: string;
password: string; firstname: string; lastname: string; email: string; city?: string; country?:
string; lang?: string; timezone?: string; idnumber?: string; } @Injectable() export class
UsersService { constructor(private moodle: MoodleService) {} /** Crear uno o varios usuarios
*/ async createUsers(users: CreateUserDto[]): Promise<{ id: number; username: string }[]> {
return this.moodle.call('core_user_create_users', { users }); } /** Buscar usuarios por
criterio */ async getUsers(field: string, values: string[]): Promise { const result: any =
await this.moodle.call('core_user_get_users_by_field', { field, values, }); return result; }
/** Actualizar datos de usuarios */ async updateUsers(users: Partial[]): Promise { await
this.moodle.call('core_user_update_users', { users }); } /** Eliminar/suspender usuarios */
async deleteUsers(userids: number[]): Promise { await
this.moodle.call('core_user_delete_users', { userids }); } /** Buscar usuario por email
(helper) */ async findByEmail(email: string): Promise { const users = await
this.getUsers('email', [email]); return users.length > 0 ? users[0] : null; } }
```

C.4 — Módulo de Cursos

moodle/courses.service.ts

```
import { Injectable } from '@nestjs/common'; import { MoodleService } from '../moodle.service';
export interface CreateCourseDto { fullname: string; shortname: string; categoryid: number;
summary?: string; format?: string; startdate?: number; enddate?: number; visible?: number;
maxbytes?: number; } @Injectable() export class CoursesService { constructor(private moodle:
MoodleService) {} /** Obtener todos los cursos o filtrar por IDs */ async getCourses(ids?:
number[]): Promise { const options = ids ? { ids } : {}; return
this.moodle.call('core_course_get_courses', { options }); } /** Buscar cursos por campo */
async getCourseByField(field: string, value: string): Promise { return
this.moodle.call('core_course_get_courses_by_field', { field, value }); } /** Buscar cursos
por texto */ async searchCourses(query: string, page = 0, perpage = 20): Promise { return
this.moodle.call('core_course_search_courses', { criterioname: 'search', criteriaval: query,
page, perpage, }); } /** Crear cursos */ async createCourses(courses: CreateCourseDto[]):
Promise { return this.moodle.call('core_course_create_courses', { courses }); } /** Obtener
contenido (secciones y actividades) de un curso */ async getCourseContents(courseid: number):
Promise { return this.moodle.call('core_course_get_contents', { courseid }); } }
```

C.5 — Módulo de Matrículas

moodle/enrolment.service.ts

```
import { Injectable } from '@nestjs/common'; import { MoodleService } from '../moodle.service';
export enum MoodleRole { MANAGER = 1, COURSE_CREATOR = 2, TEACHER = 3, NON_EDITING_TEACHER = 4,
STUDENT = 5, GUEST = 6, } @Injectable() export class EnrolmentService { constructor(private
moodle: MoodleService) {} /** Matricular un usuario en un curso */ async enrolUser( userid:
number, courseid: number, roleid: MoodleRole = MoodleRole.STUDENT, timestart = 0, timeend = 0,
): Promise { await this.moodle.call('enrol_manual_enrol_users', { enrolments: [{ roleid,
userid, courseid, timestart, timeend }], }); } /** Desmatricular un usuario */ async
unenrolUser(userid: number, courseid: number): Promise { await
this.moodle.call('enrol_manual_unenrol_users', { enrolments: [{ userid, courseid }], }); } /**
Obtener usuarios matriculados en un curso */ async getEnrolledUsers(courseid: number): Promise
{ return this.moodle.call('core_enrol_get_enrolled_users', { courseid }); } /** Obtener cursos
de un usuario */ async getUserCourses(userid: number): Promise { return
this.moodle.call('core_enrol_get_users_courses', { userid }); } }
```

C.6 — Módulo de Calificaciones

moodle/grades.service.ts

```
import { Injectable } from '@nestjs/common'; import { MoodleService } from '../moodle.service';
@Injectable() export class GradesService { constructor(private moodle: MoodleService) {} /**
Obtener calificaciones de un usuario en un curso */ async getGradeItems(courseid: number,
userid: number): Promise { return this.moodle.call('gradereport_user_get_grade_items', {
courseid, userid, groupid: 0, }); } /** Obtener tabla completa de calificaciones */ async
getGradesTable(courseid: number, userid: number): Promise { return
this.moodle.call('gradereport_user_get_grades_table', { courseid, userid, groupid: 0, }); }
/** Actualizar calificación en una actividad */ async updateGrade( courseid: number,
component: string, // ej: 'mod_assign' activityid: number, // ID de la actividad itemnumber:
number, grades: { studentid: number; grade: number; attemptnumber?: number }[], ): Promise {
await this.moodle.call('core_grades_update_grades', { source: 'external', courseid, component,
activityid, itemnumber, grades, }); } }
```

C.7 — Módulo de Mensajería

moodle/messaging.service.ts

```
import { Injectable } from '@nestjs/common'; import { MoodleService } from '../moodle.service';
@Injectable() export class MessagingService { constructor(private moodle: MoodleService) {}
/** Enviar mensaje privado a un usuario */ async sendMessage(touserid: number, text: string):
Promise { return this.moodle.call('core_message_send_messages', { messages: [{ touserid, text,
textformat: 0 }], }); } /** Enviar mensaje a múltiples usuarios */ async sendBulkMessages(
userids: number[], text: string, ): Promise { const messages = userids.map(touserid => ({
touserid, text, textformat: 0 })); return this.moodle.call('core_message_send_messages', {
messages }); } /** Obtener mensajes de un usuario */ async getMessages(userid: number, type =
'inbox'): Promise { return this.moodle.call('core_message_get_messages', { useridto: type ===
'inbox' ? userid : 0, useridfrom: type === 'sent' ? userid : 0, type, read: 0, newestfirst: 1,
limitfrom: 0, limitnum: 50, }); } }
```

C.8 — Manejo de Errores con Filtros de Excepción

moodle/moodle-exception.filter.ts

```
import { ExceptionFilter, Catch, ArgumentsHost, HttpException, HttpStatus, Logger, } from
'@nestjs/common'; import { Response } from 'express'; @Catch() export class
MoodleExceptionHandler implements ExceptionFilter { private readonly logger = new
Logger(MoodleExceptionHandler.name); catch(exception: unknown, host: ArgumentsHost) { const ctx
= host.switchToHttp(); const res = ctx.getResponse(); if (exception instanceof HttpException)
{ const status = exception.getStatus(); const body = exception.getResponse() as any;
this.logger.error(`Moodle API Error: ${JSON.stringify(body)}`); return
res.status(status).json({ statusCode: status, error: body.moodleError ?? 'MOODLE_ERROR',
message: body.message ?? 'Error al comunicarse con Moodle', timestamp: new
Date().toISOString(), }); } this.logger.error('Error inesperado:', exception);
res.status(HttpStatus.INTERNAL_SERVER_ERROR).json({ statusCode: 500, error: 'INTERNAL_ERROR',
message: 'Error interno del servidor', }); } }
```

C.9 — Interceptor de Logs

moodle/moodle-logging.interceptor.ts

```
import { Injectable, NestInterceptor, ExecutionContext, CallHandler, Logger, } from
 '@nestjs/common'; import { Observable, tap } from 'rxjs'; @Injectable() export class
 MoodleLoggingInterceptor implements NestInterceptor { private readonly logger = new
 Logger('MoodleAPI'); intercept(ctx: ExecutionContext, next: CallHandler): Observable { const
 req = ctx.switchToHttp().getRequest(); const start = Date.now(); return next.handle().pipe(
 tap({ next: () => { this.logger.log( `${req.method} ${req.url} → ${Date.now() - start}ms`, );
 }, error: (err) => { this.logger.error( `${req.method} ${req.url} ERROR ${Date.now() -
 start}ms`, err.stack, ); }, })), ); }
```

C.10 — Ejemplo Completo: Controlador REST

Controlador que expone endpoints para gestión de usuarios y matrículas:

moodle/moodle.controller.ts

[illegible]

moodle/moodle.module.ts — Módulo Completo

```
import { Module } from '@nestjs/common'; import { HttpModule } from '@nestjs/axios'; import { MoodleService } from './moodle.service'; import { UsersService } from './users.service'; import { CoursesService } from './courses.service'; import { EnrolmentService } from './enrolment.service'; import { GradesService } from './grades.service'; import { MessagingService } from './messaging.service'; import { MoodleController } from './moodle.controller'; @Module({ imports: [ HttpModule ], controllers: [ MoodleController ], providers: [ MoodleService, UsersService, CoursesService, EnrolmentService, GradesService, MessagingService, ], exports: [ MoodleService, UsersService, CoursesService, EnrolmentService, GradesService, MessagingService, ], }) export class MoodleModule {}
```